

QAOA-GPT: Hardware-Aware GPT-Assisted Circuit Generation for Quantum Optimisation

Ruairi Hogan

21432816

January 18, 2026

Name: Ruairi Hogan

Student No.: 21432816

Declaration of Authorship

I have read and understood the university's policy on plagiarism.

I declare that all material in this assessment is my own work except where there is clear acknowledgement and appropriate reference to the work of others.

Signed: 

Date: January 18, 2026

Contents

1	Introduction	3
2	Literature Review	4
2.1	Gate-Based Quantum Computing in the NISQ-era	4
2.2	Variational Quantum Algorithms	4
2.3	Quantum Approximate Optimisation Algorithm (QAOA)	4
2.4	QAOA Variants and Extensions	7
2.5	Optimisation Challenges in QAOA	9
2.6	Machine Learning and AI-Assisted Quantum Optimisation	10
2.7	GPT-Assisted Frameworks and QAOA-GPT	11
3	Problem Definition and Project Aims	11
3.1	Problem Definition	11
3.2	Project Aims	12
4	Methodology	13
4.1	Overview	13
4.2	QAOA-GPT Framework	14
5	Progress to Date and Preliminary Results	17
5.1	Implementation of Standard QAOA Circuits	17
5.2	Building QAOA-GPT Framework	18
6	Ethics and Sustainability	21
6.1	Ethical Considerations	21
6.2	Sustainability Considerations	21
7	Project Plan and Timeline	22
8	Conclusion	23

1 Introduction

Quantum computing has become an increasingly relevant field in the decades-long quest to develop a fault-tolerant, low-error machine that can solve certain problems asymptotically faster than classical computers available today. Presently, quantum computing architectures are divided into a few large classes, with gate-based approaches being one of the most dominant near-term models [1]. Gate-based quantum models use a sequence of quantum gates (unitary operations) to manipulate qubits. This reflects very similarly to how classical computers use logic gates on bits. Various methods have been proposed that are designed to run on a gate-based architecture, which heuristically solves large sets of combinatorial optimisation problems. There is considerable hope for this architecture to outperform classical computers.

With current quantum computers not yet reliable enough to provide an asymptotic quantum advantage over classical computers, algorithm engineering is crucial. Accounting for the constraints imposed by these computers, optimisation-based or learning-based approaches are widely used. A common method used by these algorithms is a classical-quantum hybrid approach. This is where a classical optimiser is used along with a quantum processor to assist in dividing computational tasks between the two machines to exploit their strengths while mitigating their respective weaknesses.

The quantum approximate optimisation algorithm (QAOA) was introduced to heuristically solve combinatorial optimisation problems [2]. It consists of layering quantum circuits to interfere with the qubit states to get a target state from the circuit. A classical optimiser is then used to optimise parameters and obtain a more accurate final state from the quantum circuit.

An emerging variant of QAOA is QAOA-GPT. This is a GPT-assisted framework for the parametrisation or optimisation of QAOA. It leverages a generative pre-trained transformer (GPT) for circuit construction using sequence modelling and pattern recognition. This project aims to build a QAOA-GPT framework and generate accurate quantum circuits with less online optimisation cost than iterative QAOA approaches.

2 Literature Review

2.1 Gate-Based Quantum Computing in the NISQ-era

The current quantum machines operate in what is coined as the Noisy Intermediate-Scale Quantum (NISQ) era. NISQ computing has achieved many significant advancements, which bring us closer to the realisation of universal fault-tolerant quantum computers. These are characterised by devices containing tens to hundreds of qubits but are not protected by full quantum error correction. While these advancements make for very powerful and hopeful results, these devices are limited by noise, circuit depth and finite qubit counts [3].

2.2 Variational Quantum Algorithms

Variational quantum algorithms (VQAs) have been proposed as a leading strategy in solving combinatorial optimisation problems on NISQ devices. VQAs use parametrised quantum circuits, which allow the outsourcing of parameter optimisation to a classical optimiser. This hybrid loop is iterated until convergence, allowing the classical and quantum components to jointly search for an approximate solution to the target problem. This is an added advantage as it keeps the quantum circuit depth shallow and therefore mitigates noise while also reducing the overhead on the quantum processor itself. Optimisers that leverage gradient-free and stochastic methods are commonly used. They are robust to noise but may require a large number of function evaluations. As a result, the overall performance of VQAs is strongly influenced by circuit structure and parameter initialisation [4].

2.3 Quantum Approximate Optimisation Algorithm (QAOA)

2.3.1 Algorithm Structure

The quantum approximate optimisation algorithm (QAOA) is a quantum-classical hybrid variational algorithm that is designed to solve combinatorial optimisation problems. QAOA is a promising step for near-term quantum applications, however there exist many limitations in heuristically selecting parameters and Hamiltonian terms. Numerous variants and

optimisation strategies have been proposed to enhance both the understanding and practical deployment of QAOA, especially in addressing challenges associated with the outer-loop classical optimisation [4, 5].

In QAOA, the optimisation problem is encoded in a cost Hamiltonian whose expectation value is minimised. The algorithm is created by alternatively applying unitaries generated by the cost Hamiltonian, which encodes the objective function, and the mixer Hamiltonian ($H_B = \sum_{i=1}^n X_i$, where X_i is the Pauli- X operator), which moves the probability amplitude between different computational basis states. The function of the mixer Hamiltonian is to promote exploration of the solution space. Iteratively, the quantum circuit is executed, evaluated, and the classical optimiser updates the variational parameters to improve the result.

The expressivity of QAOA is controlled by a parameter p , the circuit depth. This determines the number of alternating cost and mixer layers that are applied to the circuit. The circuit depth scales the computational cost polynomially, requiring careful algorithmic optimisation, typically achieved through a heuristic solution [6]. The QAOA procedure starts from the uniform superposition state $|+\rangle^{\otimes n}$ and proceeds by sequentially applying p layers of alternating unitary transformations corresponding to the problem and mixing Hamiltonians. These unitaries are defined as

$$U_P(\gamma_i) = e^{-i\gamma_i H_C} \quad \text{and} \quad U_M(\beta_i) = e^{-i\beta_i H_B}, \quad (1)$$

respectively.

After the application of p such layers, the resulting QAOA state is given by

$$|\gamma, \beta\rangle = U_M(\beta_p)U_P(\gamma_p) \cdots U_M(\beta_1)U_P(\gamma_1)|+\rangle^{\otimes n}, \quad (2)$$

where $\gamma = \{\gamma_1, \dots, \gamma_p\}$ and $\beta = \{\beta_1, \dots, \beta_p\}$ denote the corresponding sets of variational parameters.

The cost Hamiltonian expectation value is evaluated by the formula

$$\langle H_C \rangle = \langle \gamma, \beta | H_C | \gamma, \beta \rangle. \quad (3)$$

For a given depth, the QAOA state can be written as

$$|\psi(\gamma, \beta)\rangle = \prod_{k=1}^p e^{-i\beta_k H_M} e^{-i\gamma_k H_C} |+\rangle^{\otimes n} \quad (4)$$

2.3.2 Problem Formulation: QUBO and MaxCut

In the context of QAOA, combinatorial optimisation problems are commonly expressed in the form of a Quadratic Unconstrained Binary Optimisation (QUBO) problem. This seeks to minimise an objective function in the form

$$\min_{\mathbf{x} \in \{0,1\}^n} \mathbf{x}^T Q \mathbf{x} \quad (5)$$

Where Q is a real-valued matrix that encodes interactions between binary variables [7]. This form of optimisation problems is well-suited to quantum machines, as they admit a direct mapping to equivalent Ising Hamiltonians defined on qubit systems. A canonical example used in QAOA is the MaxCut problem. The objective of this is to cut the vertices of a graph of form $G = (V, E)$, where V is the set of vertices and E is the set of edges, such that the sum of weights of edges crossing the partition (cut) is maximised [8].

For a weighted graph, the MaxCut cost Hamiltonian can be written as

$$H_C = \sum_{(i,j) \in E} \frac{w_{ij}}{2} (I - Z_i Z_j) \quad (6)$$

where w_{ij} denotes the edge weight and Z_i is the Pauli-Z operator acting on qubit i . Maximising the cut value therefore minimises the expectation value of H_C . MaxCut is widely used as a benchmark problem for QAOA due to its well-defined QUBO formulation, and has well-understood classical optima for small instances. MaxCut will be used as the problem set for QAOA algorithms to solve in this project.

2.3.3 Performance Characteristics

QAOA performance is typically evaluated using the expected energy of the cost Hamiltonian or the approximation ratio relative to the optimal solution. The expressivity and performance can improve with increased p ; however, studies have shown that the performance gains diminish rapidly beyond small depths [5]. The limitations for the performance of QAOA pertain to the trade-off of computational cost and circuit depth, the effectiveness of the classical optimiser, and the noise interference of quantum processors in the NISQ-era.

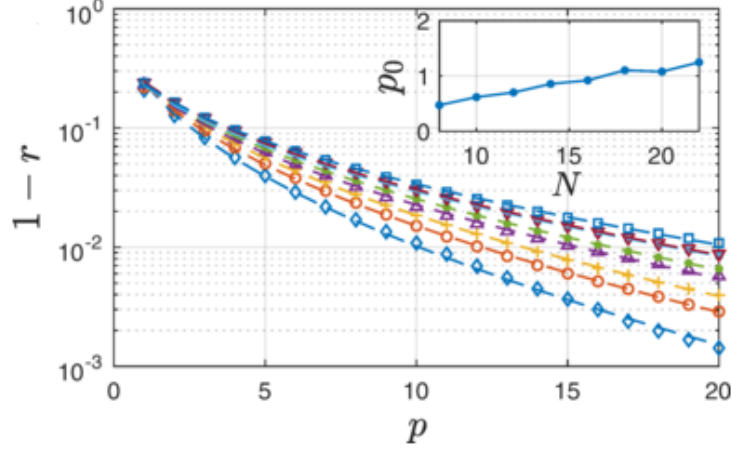


Figure 1: *Performance of QAOA as a function of circuit depth p , illustrating diminishing improvements in approximation quality at larger depths [5]*

2.4 QAOA Variants and Extensions

2.4.1 Alternative Mixers

In standard QAOA, the mixer Hamiltonian is usually chosen as a uniform transverse-field operator, $H_M = \sum_i X_i$, as it ensures the traversal of the entire solution space and aims to prevent the algorithm from getting stuck in local minima. The mixer operator is the most critical component of QAOA [9]. The purpose of alternative mixer approaches is to modify this component by replacing the standard, uniform mixer with a mixer operator proposed to improve searching efficiency. This approach typically still retains the standard QAOA structure of alternating cost and mixer Hamiltonians in the quantum circuit. Some examples of alternative mixer QAOA include XY-type mixers, graph-aware mixers and quantum approximate Bayesian optimisation algorithm (QABOA) [9, 10].

The unconstrained mixing can drive the quantum state into invalid or low-quality regions in the solution space, thus reducing the efficiency of converging at a suitable solution. While these mixers can improve performance, there are limitations. Constructing effective mixers typically requires domain knowledge. Additionally, some alternative mixers can introduce more overhead on hardware with limited connectivity by introducing more complex interactions and increasing circuit depth. [11].

2.4.2 Warm-Start Approaches

Warm-start approaches are complementary strategies for improving the practicality of variational quantum optimisation in the NISQ-era.

Warm-start methods use classical approximate solutions to initialise the quantum state with a bias towards searching in favourable regions of the solution space. This often accelerates convergence when reliable classical heuristics are known [12].

This strategy can be used in relation to this project because for extremely complex problems with large datasets, the GPT will have to undergo huge amounts of rigorous training to achieve a circuit that can perform accurately. When the demand for the energy and computational cost of this training is infeasible, the model can be used to generate operators and parameters that serve as a warm start for an alternative QAOA algorithm.

2.4.3 ADAPT-QAOA

There is evidence suggesting QAOA may provide a significant advantage in heuristically solving complex problems and is computationally universal [10, 13]. The performance improves with the number of alternating layers in the Ansatz. This performance increase is limited by the coherence times in current and near-term quantum processors. Short-depth QAOA is sometimes deemed an ad hoc Ansatz as the results should not be expected to perform optimally or even decently [14]. The proposed adaptive derivative assembled problem-tailored quantum approximate optimisation algorithm (ADAPT-QAOA) is an iterative version of QAOA with aims to improve performance with these limitations. The ADAPT-QAOA algorithm grows the Ansatz two operators at a time by using a gradient criterion, which selects the mixer from a predefined operator pool. [14]. At each iteration k , the operators A_j are evaluated using the energy gradient from a predefined mixer pool.

$$g_j = \frac{\partial E}{\partial \beta_j} = -i \langle \psi(\boldsymbol{\beta}^{(k-1)}) | e^{i\theta H_C} [H_C, A_j] e^{-i\theta H_C} | \psi(\boldsymbol{\beta}^{(k-1)}) \rangle. \quad (7)$$

where H_C denotes the cost Hamiltonian, θ is the initial variational parameter associated with the problem Hamiltonian, and g_j represents the energy gradient corresponding to the

j -th operator in the operator pool. The operator exhibiting the strongest gradient response is then selected, the variational parameters are re-optimised to minimise the cost function $\langle \psi^{(k)} | H_C | \psi^{(k)} \rangle$.

ADAPT-QAOA addresses issues with manual tuning of layers and operator structure, as poor choices can lead to poor or inefficient convergence. This method allows the circuit to adapt to the structure of the specific optimisation instance, aiming to improve solution quality with fewer layers than a standard QAOA circuit [15]. The relevance of ADAPT-QAOA for QAOA-GPT is its ability to produce variable-length problem- dependent circuit sequences that encode heuristics about effective operator selection. This structured data allows a generative model to learn patterns in circuit construction without the computational expense of ADAPT-QAOA.

2.5 Optimisation Challenges in QAOA

VQAs such as QAOA might be the key to obtaining a near-term advantage in quantum computing, but they face many important challenges [4]. QAOA is characterised by highly non-convex optimisation landscapes arising from the interaction between the circuit structure, unitaries and operators. The resulting objective functions therefore exhibit many local minima, flat regions and sharp gradients, making these optimisations sensitive to many mutable parameters. As the complexity of the problem increases, the depth of the circuit increases and these landscapes become increasingly more complex. Algorithms such as ADAPT-QAOA have aimed to address these limitations, and while these solutions help with circuit depth, as the problem size increases, the iterative process puts a lot of computational cost on the classical optimiser [15, 16]. Leveraging an approach using a pre-trained transformer, namely QAOA-GPT, can shift the computational cost from online optimisation to offline training of the model. This will reduce computational cost during circuit creation [16] while also learning patterns to help the circuits perform better with noise interference. There still exists a challenge with scalability with this approach, but the bottleneck limit is increased.

2.6 Machine Learning and AI-Assisted Quantum Optimisation

2.6.1 ML for Parameter Initialisation and Optimisation

Machine learning (ML) has become increasingly more explored as a means of improving performance in VQAs. Early approaches focus on using ML for parameter initialisation. Approaches like warm-start QAOA leverage this technique, where models are trained to predict favourable starting parameters based on problem features as discussed in Section 2.4.2. This aims to initialise a quantum state to explore favourable regions of the solution space [12].

2.6.2 RL for Ansatz/Circuit Construction

Reinforcement Learning (RL) methods have been proposed to address the problem of Ansatz and circuit creation. This is an approach where an agent sequentially selects gates and/or operators, receiving feedback based on the improvement of the objective function or circuit efficiency. RL-based approaches have demonstrated their ability to discover problem-specific circuit structures. However, they are hard to implement on NISQ machines, they are extremely hard to simulate, and often require extensive interaction with hardware, which leads to high training cost and limited scalability [17].

2.6.3 Sequence Models and Transformer-Based Approaches

Sequence models such as transformers have been proposed more recently as a natural fit for representing quantum circuits. Parametrised circuits such as QAOA can be expressed as a sequence of gates or Hamiltonian terms. This allows a transformer model to learn the circuits as a series of tokens. This series of tokens allows the pre-trained models to learn statistical regularities in circuit constructions across many problem instances, which reduces cost where online evaluation is expensive. There still exist challenges with data scarcity and risk of overfitting to narrow problem classes, but this potential advantage directly motivates the QAOA-GPT framework [18].

2.7 GPT-Assisted Frameworks and QAOA-GPT

QAOA-GPT is a GPT-assisted framework for QAOA. GPT is a semi-supervised learning framework that supports the standard QAOA workflow. The model operates at the level of circuit design by predicting parameters for a given problem using pattern recognition. Using the GPT to synthesise the problem-specific quantum circuits, it removes the need for iterative optimisation of algorithms.

As the circuit depth increases or additional operator choices are introduced, the search space for the combinatorial problem grows exponentially, resulting in an increasing need for processing cost from a classical optimiser. QAOA-GPT exchanges the computational cost from online optimisation to offline learning, thereby allowing larger combinatorial optimisation problems to be accessible to approximate solution [18].

Although it may seem paradoxical for a classical model to assist with problems that resist classical solution methods, the GPT component does not perform the optimisation itself. Rather, it generates quantum circuits whose execution on quantum hardware carries out the computationally intensive task.

The ADAPT-QAOA approach for QAOA-GPT trains the GPT on a data set generated from the ADAPT-QAOA algorithm. This algorithm enhances the accuracy by adaptively constructing the variational circuit [14]. Identified research gaps in QAOA-GPT include **Hardware-aware GPT for Circuit Creation** and **QAOA-GPT for Different Problem Classes** other than MaxCut.

3 Problem Definition and Project Aims

3.1 Problem Definition

The practical performance of QAOA is highly dependent on circuit structure, parameter initialisation and hardware compatibility. QAOA is a flexible framework for approximate optimisation, however the circuit configurations needed for effective optimisation are greatly dependent on problem-specific heuristics that scale poorly. As a result, the process of designing efficient QAOA circuits remains a significant bottleneck in the NISQ-era of gate-based

quantum computing.

Existing approaches to QAOA are designed to address aspects of this challenge. Standard fixed-structure QAOA offers simplicity and low overhead but lacks the ability to adapt to specific problems and hardware features. Adaptive variants, such as ADAPT-QAOA, offer more expressivity and accuracy [14] but substantially increase search and optimisation overhead. In addition, algorithms that perform well on specific problem classes often do not generalise across different platforms or hardware, leaving a gap in algorithmic design and practical deployment. Due to the shortcomings of NISQ-era quantum computers, the algorithms designed must contain a certain aspect of generalisation which will allow them to build circuits that can adapt to hardware noise and decoherence across different hardware platforms. This highlights the need for optimisation frameworks that incorporate hardware constraints into the design process in a manner that preserves hardware-agnostic capability.

3.2 Project Aims

As mentioned in Section 2.7, QAOA-GPT offers an approach to addressing the bottlenecks of QAOA in the NISQ era. Given that QAOA circuits are built with a repeated structure of cost and mixer layers, this allows a GPT to evaluate global relationships within the circuit. The pre-trained model can learn these regularities and generate circuits for given optimisation problems.

A prominent bottleneck in the NISQ-era processors is hardware constraints. This leads to an increasing importance on creating algorithms that can leverage hardware information to improve performance [3]. Incorporating hardware constraints to QAOA-GPT is a natural extension of the approach and can hopefully adapt circuit generation depending on the hardware platform on which the circuit will be implemented.

This project aims to investigate and enhance a GPT-assisted QAOA framework by exploring the integration of hardware-aware circuit generation to allow the model to generate accurate quantum circuits and to reduce the number of expensive online classical optimisation iterations relative to standard iterative QAOA parameter optimisation.

3.2.1 Specific Objectives

- Implement a baseline QAOA-GPT framework for QAOA circuit generation.
- Improve the training dataset by adding FEATHER graph embeddings to training tokens.
- Improve the QAOA-GPT framework to achieve an elevated average approximation ratio.
- Incorporate hardware-related constraints into the GPT-assisted framework.
- Compare hardware-aware and hardware-independent QAOA-GPT performance.
- Identify trade-offs between the generated circuits' expressivity and hardware feasibility
- Compare standard iterative QAOA workflows with QAOA-GPT in terms of computational cost and practical execution time.

4 Methodology

4.1 Overview

This project adopts an experimental evaluation methodology to assess the impact of QAOA-GPT for circuit design. A fixed QAOA-GPT framework is used as a baseline throughout. The project leverages a GPT to directly synthesise quantum circuits to solve quadrature unconstrained binary optimisation problems. The model is pre-trained on a synthetic dataset generated using the ADAPT-QAOA approach. This pre-trained model is then demonstrated by attempting to solve the MaxCut problem on graphs. MaxCut is chosen as the benchmark for QAOA as discussed in Section 2.3.2. Performance will be evaluated between hardware-independent and hardware-aware forms of GPT guidance in order to assess the model's ability to generate appropriate QAOA circuits under differing hardware constraints. All experiments presented in this work are conducted in simulation to ensure reproducibility and systematic control over problem instances, with execution on physical IBM Quantum hardware reserved exclusively for validation of the GPT-generated circuits using the real hardware constraints.

4.2 QAOA-GPT Framework

4.2.1 Graph Embeddings (FEATHER)

Random connected graphs are generated (with reproducible seeds) without isolated nodes or multiple components for these experiments. Each edge ij is assigned a random weight w_{ij} . FEATHER is a non-parametric graph embedding based on characteristic functions which represent local distributions of node features [18]. For each node u , the r -scale random walk weighted characteristic function is computed:

$$\phi_u(\theta, r) = \sum_{w \in V} \hat{A}_{u,w}^r e^{i\theta x_w} \quad (8)$$

where $\hat{A} = D^{-1}A$ is the normalised adjacency matrix. $\hat{A}_{u,w}^r$ denotes the probability of reaching node w from u in r steps via a random walk. This function is evaluated for a set of d frequencies $\theta \in \Theta$. This forms a complex-valued embedding that records heuristics of the higher-order structure and attribute distribution around each node. After the information is retrieved for each node, the graph-level representations are then constructed using the mean (pooling) node-level embeddings, giving strong descriptors that are invariant to graph isomorphisms and robust to local perturbations in node or edge features. FEATHER scales linearly with graph size and supports multiple input features, highlighting its benefit for GPT training.

4.2.2 Circuit Generation (ADAPT-QAOA)

The circuits for training the model are constructed from randomly generated graphs, solved using ADAPT-QAOA. These circuits are constructed iteratively as described in Section 2.4.3. Many stopping criteria are used for ADAPT-QAOA circuit generation, with this framework using a target approximation ratio to determine the merit of each circuit.

$$\alpha = \frac{\langle \psi(\gamma, \beta) | H_C | \psi(\gamma, \beta) \rangle}{\text{OPT}(G)}, \quad (9)$$

where H_C denotes the cost Hamiltonian derived from the Max-Cut instance, and $\text{OPT}(G)$ represents the best-known solution value obtained using classical heuristic methods. The number of qubits considered in the simulation is limited by computational cost and thus is relatively small. As a result, the classical heuristic methods achieve near-optimal solutions for the MaxCut problem.

4.2.3 Training Set Construction

The graphs are represented by a weighted edge list. Each circuit is expressed as a sequence of layers, constructed as tokens representing operator index o_k and optimised parameters γ_k, β_k for a layer k . This data is then stored together in a structured list of tokens that is used to train the GPT. The FEATHER embeddings are broadcast across all tokens. The token set will follow this format:

$$\begin{aligned} \mathcal{S} = & \langle \text{bos} \rangle (e_{i_1 j_1}, w_{i_1 j_1}) (e_{i_2 j_2}, w_{i_2 j_2}) \dots (e_{i_m j_m}, w_{i_m j_m}) \langle \text{end_of_graph} \rangle \\ & \langle \text{new_layer}_1 \rangle (o_1, \gamma_1, \beta_1) \langle \text{new_layer}_2 \rangle (o_2, \gamma_2, \beta_2) \dots \langle \text{new_layer}_p \rangle (o_p, \gamma_p, \beta_p) \end{aligned} \quad (10)$$

where $e_{ij} = (i, j)$ represents an edge between vertices i and j , w_{ij} denotes the weight associated with edge (i, j) in the graph, m is the total number of edges, o_k is the operator selected from a predefined operator pool, γ_k and β_k are the variational parameters associated with the cost and mixer operators at layer k , respectively, and p is the total number of circuit layers

4.2.4 Training the Model

This finalised training set is used to train a decoder-only transformer model. A modified version of nanoGPT [19] is utilised following the QAOA-GPT framework of Tyagin et al. [18]. The model is trained on the graph-circuit token list, and the graph embeddings are broadcast across all tokens. The training pipeline is represented in the following graph:

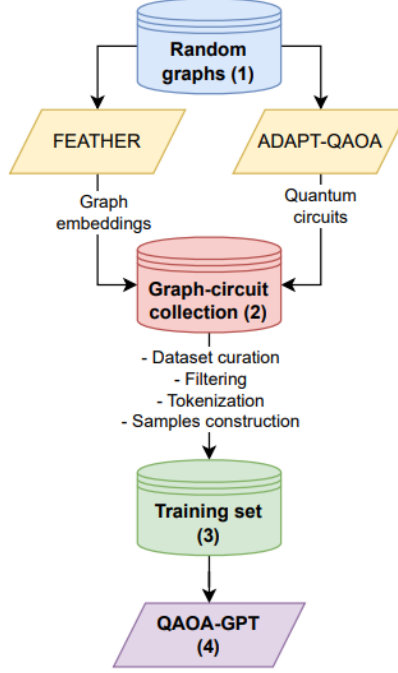


Figure 2: *QAOA-GPT Training Pipeline [18]*

4.2.5 Hardware-Aware GPT Extension

The hardware-aware GPT extension aims to improve circuit generation for practical implementation. Simulable hardware characteristics such as maximum allowable circuit depth, single and two-qubit gate counts, and qubit connectivity, gate and readout error rates will be represented as structured input features and provided to the model alongside the graph and circuit tokens. These features will be treated as auxiliary context rather than as part of the auto-regressively generated token sequence, allowing hardware constraints to be incorporated without modifying the underlying sequence structure.

4.2.6 Evaluating Performance

The performance of the model will be evaluated by selecting a list of graph tokens, and in further development, hardware characteristics, for the model to generate an appropriate quantum circuit. This circuit will be simulated using the Python module Qiskit [20], and the approximation ratio of the solution will determine the performance. In further extension, the circuit can be tested using the IBM cloud computing platform [21] on a real gate-based

quantum computer using the real hardware characteristics listed for the specific machine. This methodology enables a controlled evaluation of QAOA-GPT and its hardware-aware extension.

5 Progress to Date and Preliminary Results

5.1 Implementation of Standard QAOA Circuits

As an initial step, a baseline implementation of standard QAOA was developed to gain a reference for performance and a baseline for computational time required to achieve a significant approximation ratio. The algorithm was simulated using IBM's Python package, Qiskit [20], with an ideal circuit. A script to simulate the quantum computer and a separate script to act as the classical optimiser were created. The circuit was limited to a depth, p , of 3 layers. IBM's gradient-free COBYLA optimiser was used as it was found to produce the best results with the quickest convergence time. This hybrid simulation was run for 50 iterations to solve the MaxCut problem on a randomly generated graph with 15 vertices, simulating 15 qubits. For each iteration, the quantum simulation produced 512 shots. The mean cut-value of these was given to the classical optimiser to improve the parameters. A diagram of the simulation is displayed in fig.3 below.

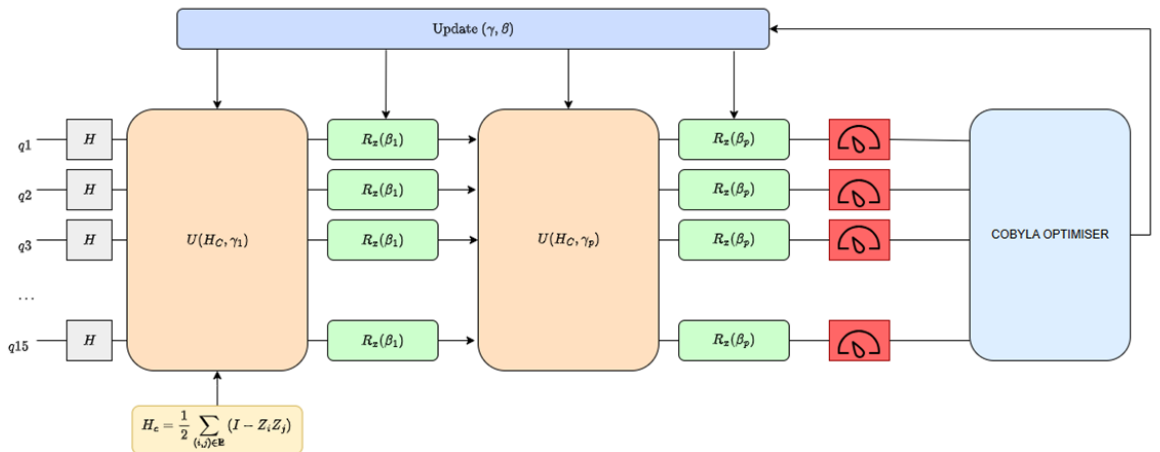


Figure 3: *Diagram of Standard QAOA Quantum-Classical Hybrid System*

The standard QAOA performed well and found the optimal MaxCut value in the majority

of the simulations. However, this baseline implementation of QAOA had an average run time of approximately 50 seconds for 50 iterations. Considering the small number of qubits simulated and a small circuit depth, this was a considerable amount of time. In addition, the algorithm frequently fails to escape local optima, which causes some simulations to perform poorly, as seen on fig.4 and fig.5, which were run back to back, for the same graph.

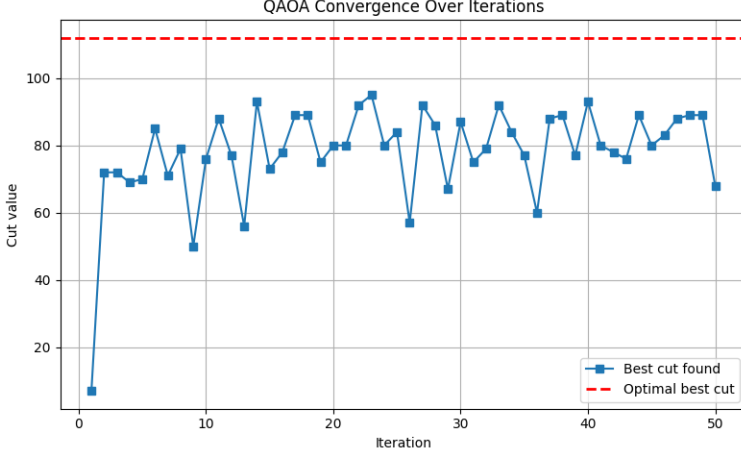


Figure 4: *Poor convergence behaviour of standard QAOA*

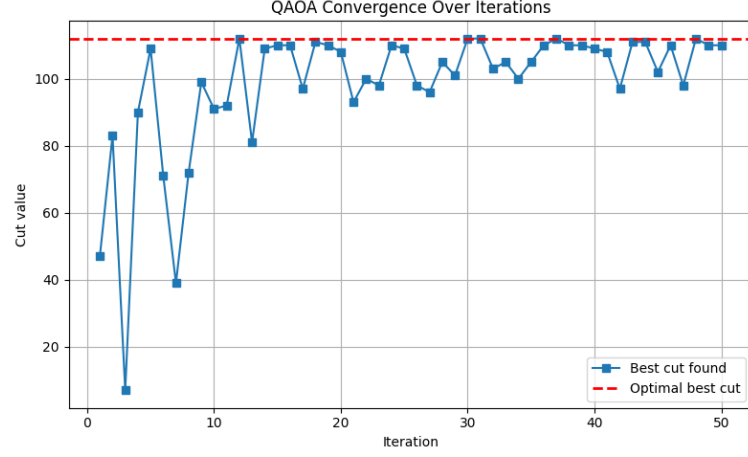


Figure 5: *Optimal convergence behaviour of standard QAOA*

5.2 Building QAOA-GPT Framework

5.2.1 Dataset Generation for GPT Training

As discussed in Section 4.2, a Python script that builds ADAPT-QAOA circuits was implemented to generate a training set for the GPT model. The script generates a random graph, and the ADAPT-QAOA algorithm is performed to find the solution to the MaxCut problem on the given graph. The quantum circuit is simulated again using IBM’s Qiskit package [20]. Graphs were generated with 7 vertices and, therefore, 7 qubits were simulated to solve the MaxCut problem. The circuits were limited to a depth of 9 layers due to computational cost and the diminishing gains of increasing p beyond a small number. The algorithm was run 4 times on each graph with a different value used to initialise the variational parameter γ_0 of the problem Hamiltonian. This is the *initial* variational parameter, used to evaluate operator gradients during the selection step; it sets the scale of the gradient probe. The potential values of γ_0 were $[0.01, 0.1, 0.5, 1.0]$. The stopping criterion for circuit generation was when the circuit reached a target approximation ratio (AR) of 0.97. Circuits generated to the target

AR were then written to a .json file in the format mentioned in Section 4.2.3. A dataset of 9857 accurate circuits was generated.

5.2.2 Data Pre-processing

A corpus and vocabulary file were made from the resulting .json file, and a vocabulary file to define the discrete token set. The training data was then separated, with 10% being used for validation and 90% for training.

5.2.3 Training of the GPT

The model was then trained on the data generated. FEATHER embeddings were not used. Across 16'000 iterations, the loss of the model during training decreased from 0.5224 to 0.2531, which indicates the model's increasing success at predicting the next token in circuit generation.

5.2.4 Results From Circuit Generation

The trained model was then evaluated. A Python script was used to generate 1000 random graphs with 7 vertices. The graphs were input to the GPT, and a resulting 1000 circuits were generated. Using Qiskit [20], the circuits were run through an ideal simulation to find their solution. A brute force algorithm was implemented to find the optimal value for the MaxCut problem for the respective graph. The results from this test are shown in Table 1.

The results indicate that the model was significantly accurate in generating quantum circuits for these MaxCut problems. Table 1 summarises the performance of the GPT-generated circuits. A high success rate of 98.10% was observed; the circuits would be classified as invalid if they were missing parameters, edge weights or tags. The test shows a mean approximation ratio of 0.8994 and relatively low variability. While only one circuit achieved an optimal solution, the majority of valid circuits achieved adequate performance. The total time to generate the 1000 circuits was 23 minutes and 21 seconds, equating to a mean time for generating a single circuit of 1.401 seconds. This is a significant improvement over standard QAOA and shows

Metric	Value
Total circuits evaluated	1000
Valid circuits	981
Failed circuits	19
Success rate	98.10%
Mean approximation ratio	0.8994
Standard deviation	0.0669
Coefficient of variation	0.0744
Best-performing circuit	1.0000
Worst valid circuit	0.6141
Number of perfect circuits	1

Table 1: *Summary of QAOA-GPT Circuit Evaluation Results*

the benefit of QAOA-GPT for reducing exploration cost, even if it doesn’t always improve final performance.

5.2.5 Further Optimisations of the GPT

Currently, the results indicate the model can perform adequately in circuit generation. However, there are a few improvements that can increase the performance of the model.

- **Training on a GPU:** Due to the computational performance of a CPU, the generation of training data and the configuration of training the model are limited. Using a GPU for data creation and training will allow for a higher qubit count to be simulated, increased model complexity and faster training time. This will produce more efficiently and accurately.
- **Adding FEATHER Embeddings:** As mentioned in Section 4.2.1, FEATHER graph embeddings will provide better performance as the model will have a better understanding of the global structural properties of the graph instead of trying to infer these purely from long sequences of edge tokens.
- **Extending the GPT framework to generate QAOA circuits for previously unseen problem sizes:** Currently, the GPT can generate circuits for 7-qubit problems and less. A potential optimisation could be adding and testing the ability of the GPT

to generate circuits for problems with previously unseen numbers of qubits

6 Ethics and Sustainability

6.1 Ethical Considerations

- The project is exploratory and methodological, with no real-world deployment or autonomous decision-making
- There exists a risk of over-interpreting the results generated by AI, which is mitigated by reporting controlled, simulation-based evaluations only.
- All claims are restricted to an experimental setting to avoid misleading conclusions in quantum advantages or practical capability.

6.2 Sustainability Considerations

- Classical simulations are used predominantly for research in NISQ-era quantum computers, which increases computational and energy cost for research in this field due to exponential scaling with the simulated qubit count. This project deliberately constrains problem size, circuit depth and training configurations to manage computational resource usage.
- ML-assisted circuit generation is explored as a way of reducing the repeated random exploration and expensive optimisation iterations.
- Training a lightweight model (nanoGPT [19]) and reusing the model across evaluations avoids repeated high-cost optimisation runs.
- Introducing hardware constraints can potentially reduce the production of circuits that are theoretically valid but are infeasible on realistic hardware.
- Efficient circuit design is treated as a sustainability objective, not just a performance objective.

- Although there are many benefits to leveraging a GPT in circuit design, the exponential scaling of problem complexity with qubit count will need a significant amount of energy to train the GPT. The proposed aim is that this problem will be mitigated by the ability to re-use the trained model, negating the need for expensive online optimisation in other QAOA variants.

7 Project Plan and Timeline

As mentioned in Section 3.2.1, the end objective of this project is to create a GPT-assisted framework that can generate accurate circuits for specific hardware with less practical computational cost than traditional iterative optimisation QAOA methods. The plans include adding FEATHER embedding to the training of the model, Optimisation of the model using a GPU, changing training configurations and adding a hardware-aware extension to the training process to produce hardware-specific circuits. Fig.6 displays the Gantt chart outlining the plan for the next steps of the project.

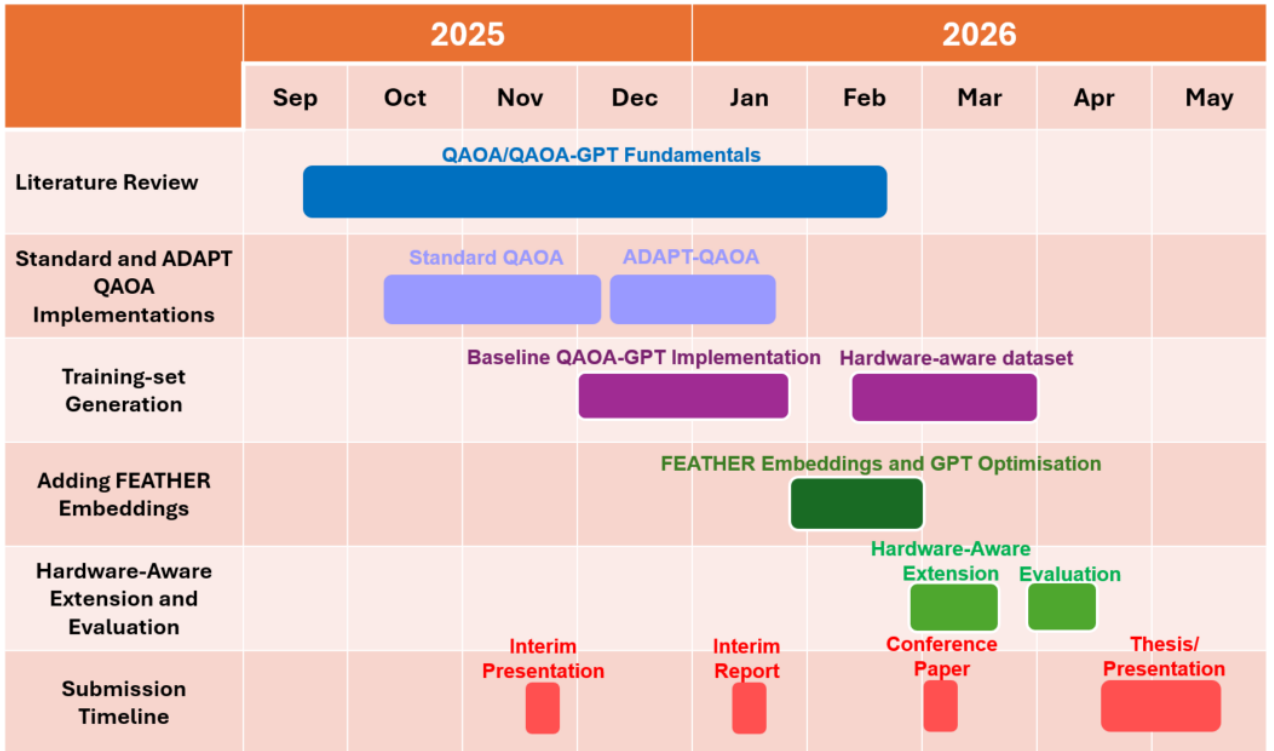


Figure 6: Gantt chart outlining the plan for this project

Fig.6 shows the progress already completed, namely building a baseline QAOA-GPT frame-

work, leaving 3 milestones to be completed before evaluation can take place. These are:

- Using FEATHER embeddings to train the model
- Optimising the model with more powerful computation
- Extending the GPT to generate hardware-specific circuits given the characteristics of the hardware.

8 Conclusion

This report has outlined background research, project aims, limitations, proposed methodology and the significant progress made in the development of a QAOA-GPT framework. The preliminary results indicate the possibility for the framework to perform to a high standard and potentially outperform various other QAOA methods on practical computational cost under the constraints of the project's feasible simulation.

References

- [1] T. Fankhauser, M. E. Solèr, R. M. Füchslin, and K. Stockinger, “Multiple query optimization using a gate-based quantum computer,” *IEEE Access*, vol. 11, 2023.
- [2] S. Hadfield, Z. Wang, B. O’Gorman, E. G. Rieffel, D. Venturelli, and R. Biswas, “From the quantum approximate optimization algorithm to a quantum alternating operator ansatz,” *Algorithms*, vol. 12, no. 2, 2019. [Online]. Available: <https://www.mdpi.com/1999-4893/12/2/34>
- [3] M. AbuGhanem and H. Eleuch, “Nisq computers: A path to quantum supremacy,” *IEEE Access*, vol. 12, 2024.
- [4] M. Cerezo, A. Arrasmith, R. Babbush, S. C. Benjamin, S. Endo, K. Fujii, J. R. McClean, K. Mitarai, X. Yuan, L. Cincio, and P. J. Coles, “Variational quantum algorithms,” *Nature Reviews Physics*, vol. 3, no. 9, pp. 625–644, 2021.
- [5] L. Zhou, S.-T. Wang, S. Choi, H. Pichler, and M. D. Lukin, “Quantum approximate optimization algorithm: Performance, mechanism, and implementation on near-term devices,” *Phys. Rev. X*, vol. 10, p. 021067, Jun 2020. [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevX.10.021067>
- [6] V. Akshay, H. Philathong, E. Campos, D. Rabinovich, I. Zacharov, X.-M. Zhang, and J. D. Biamonte, “Circuit depth scaling for quantum approximate optimization,” *Phys. Rev. A*, vol. 106, p. 042438, Oct 2022. [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevA.106.042438>
- [7] D. Volpe, G. Orlandi, and G. Turvani, “Improving the solving of optimization problems: A comprehensive review of quantum approaches,” *Quantum Reports*, vol. 7, no. 1, 2025. [Online]. Available: <https://www.mdpi.com/2624-960X/7/1/3>
- [8] F. Rendl, G. Rinaldi, and A. Wiegeler, “Solving max-cut to optimality by intersecting semidefinite and polyhedral relaxations,” *Mathematical Programming*, vol. 121, no. 2, pp. 307–335, 2010.
- [9] J. E. Kim and Y. Wang, “Quantum approximate bayesian optimization algorithms with two mixers and uncertainty quantification,” *IEEE Transactions on Quantum Engineering*, vol. 4, 2023.
- [10] Z. He, R. Shaydulin, S. Chakrabarti, D. Herman, C. Li, Y. Sun, and M. Pistoia, “Alignment between initial state and mixer improves qaoa performance for constrained optimization,” *npj Quantum Information*, vol. 9, no. 1, p. 121, 2023.
- [11] F. G. Fuchs and R. P. Bassa, “LX-mixers for QAOA: Optimal mixers restricted to subspaces and the stabilizer formalism,” *Quantum*, vol. 8, p. 1535, Nov. 2024. [Online]. Available: <https://doi.org/10.22331/q-2024-11-25-1535>
- [12] R. Zhao, T. Cheng, R. Wang, X. Fan, and H. Ma, “Artificial intelligence warm-start approach: optimizing the generalization capability of qaoa in complex energy landscapes,” *New Journal of Physics*, vol. 26, no. 5, p. 053016, may 2024. [Online]. Available: <https://doi.org/10.1088/1367-2630/ad4629>

- [13] P. K. Barkoutsos, G. Nannicini, A. Robert, I. Tavernelli, and S. Woerner, “Improving Variational Quantum Optimization using CVaR,” *Quantum*, vol. 4, p. 256, Apr. 2020. [Online]. Available: <https://doi.org/10.22331/q-2020-04-20-256>
- [14] L. Zhu, H. L. Tang, G. S. Barron, F. A. Calderon-Vargas, N. J. Mayhall, E. Barnes, and S. E. Economou, “Adaptive quantum approximate optimization algorithm for solving combinatorial problems on a quantum computer,” *Phys. Rev. Res.*, vol. 4, p. 033029, Jul 2022. [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevResearch.4.033029>
- [15] N. Yanakiev, N. Mertig, C. K. Long, and D. R. M. Arvidsson-Shukur, “Dynamic adaptive quantum approximate optimization algorithm for shallow, noise-resilient circuits,” *Phys. Rev. A*, vol. 109, p. 032420, Mar 2024. [Online]. Available: <https://link.aps.org/doi/10.1103/PhysRevA.109.032420>
- [16] L. Sunny, A. Rijal, and G. Siopsis, “Extending qaoa-gpt to higher-order quantum optimization problems,” 2025. [Online]. Available: <https://arxiv.org/abs/2511.07391>
- [17] S. Y.-C. Chen, C.-H. H. Yang, J. Qi, P.-Y. Chen, X. Ma, and H.-S. Goan, “Variational quantum circuits for deep reinforcement learning,” *IEEE Access*, vol. 8, pp. 141 007–141 024, 2020.
- [18] I. Tyagin, M. H. Farag, K. Sherbert, K. Shirali, Y. Alexeev, and I. Safro, “QAOA-GPT: Efficient Generation of Adaptive and Regular Quantum Approximate Optimization Algorithm Circuits,” in *2025 International Conference on Quantum Computing and Engineering*, 4 2025.
- [19] A. Karpathy, “nanoGPT,” <https://github.com/karpathy/nanoGPT>, 2023, gitHub repository.
- [20] IBM Quantum, “Qiskit: An Open-Source Framework for Quantum Computing,” <https://www.ibm.com/quantum/qiskit>.
- [21] —, “IBM Quantum Cloud,” <https://quantum.cloud.ibm.com/>.